2. TEE 실행 환경 프로세스

개요

Sapphire ParaTime의 핵심인 TEE(Trusted Execution Environment) 실행 환경에서

스마트 컨트랙트가 어떻게 실행되고 검증되는지 상세히 설명합니다.

2.1 TEE 아키텍처

Intel SGX 기반 구조

```
┌─────────────────────────────────────────────────────────────┐
│                          Host OS                            │
│ ┌──────────────────────────────────────────────────────────┐ │
│ │                       SGX Enclave                        │ │
│ │ ┌────────────────────────────────────────────────────┐ │ │
│ │ │                  Sapphire Runtime                  │ │ │
│ │ │ ┌──────────────────────────────────────────────┐ │ │ │
│ │ │ │                   EVM Module                 │ │ │ │
│ │ │ │                - 기밀 상태 관리                 │ │ │ │
│ │ │ │                - 암호화 프리컴파일              │ │ │ │
│ │ │ │                - 가스 계산                      │ │ │ │
│ │ │ └──────────────────────────────────────────────┘ │ │ │
```

```
| | | ┌──────────────────────────────────────────┐ | | |
| | | |                Core Modules                | | | |
| | | |                - Accounts                  | | | |
| | | |                - Consensus                 | | | |
| | | |                - Rewards                   | | | |
| | | └──────────────────────────────────────────┘ | | |
| | └──────────────────────────────────────────────┘ | |
| └────────────────────────────────────────────────────┘ |
└──────────────────────────────────────────────────────────┘
```

런타임 모듈 구성

```
type Modules = (
modules::core::Module<Config>, // 핵심 기능
modules::accounts::Module, // 계정 관리
modules::consensus::Module, // 합의 인터페이스
modules::consensus_accounts::Module, // 합의 계정
modules::rewards::Module, // 보상 시스템
modules::rofl::Module<Config>, // ROFL 지원
module_evm::Module<Config>, // EVM 실행
module_rofl_market::Module<Config>, // ROFL 마켓
);
```## 2.

2 런타임 초기화 프로세스

시작 과정

```rust
fn main() {
<sapphire_paratime::Runtime as oasis_runtime_sdk::Runtime>::start();
}
```

제네시스 상태 설정

```rust
fn genesis_state() -> <Self::Modules as sdk::module::MigrationHandler>::Genesis
(
// Core 모듈 설정
modules::core::Genesis {
parameters: modules::core::Parameters {
min_gas_price: BTreeMap::from([(Denomination::NATIVE, 100_000_000_000)]),
dynamic_min_gas_price: modules::core::DynamicMinGasPrice {
enabled: true,
target_block_gas_usage_percentage: 50,
min_price_max_change_denominator: 8,
},
max_batch_gas: 15_000_000,
```

```
max_tx_size: 128 * 1024,

max_tx_signers: 3,

max_multisig_signers: 8,

// ...

},

},

// 기타 모듈들...

)

}
```

2.3 트랜잭션 실행 프로세스

배치 처리 구성

```
const SCHEDULE_CONTROL: config::ScheduleControl = config::ScheduleControl {

initial_batch_size: 50, // 초기 배치 크기

batch_size: 50, // 표준 배치 크기

min_remaining_gas: 1_000, // 최소 잔여 가스

max_tx_count: 1_000, // 최대 트랜잭션 수

};
```

EVM 실행 환경

```
impl module_evm::Config for Config {

type AdditionalPrecompileSet = ();

const CHAIN_ID: u64 = chain_id(); // 체인 ID

const TOKEN_DENOMINATION: Denomination = Denomination::NATIVE;

const CONFIDENTIAL: bool = true; // 기밀성 활성화

}
```

2.4 기밀성 보장 메커니즘

상태 암호화

- **메모리 보호**: SGX 엔클레이브 내부에서만 평문 상태 접근 가능
- **디스크 암호화**: 영구 저장 시 모든 상태 데이터 암호화
- **네트워크 암호화**: 노드 간 통신 시 암호화된 상태만 전송

실행 격리

- **하드웨어 격리**: SGX를 통한 하드웨어 수준 격리
- **메모리 보호**: 엔클레이브 외부에서 메모리 접근 불가
- **사이드 채널 보호**: 타이밍 공격 등 사이드 채널 공격 방어

2.5 암호화 프리컴파일

랜덤 바이트 생성

```
// 주소: 0x0100000000000000000000000000000000000001
```

```
// VRF 기반 안전한 랜덤 생성
function randomBytes(uint256 numBytes, bytes memory pers)
internal view returns (bytes memory)
```

키 유도

```
// 주소: 0x0100000000000000000000000000000000000002
// X25519 ECDH 키 유도
function deriveSymmetricKey(
Curve25519PublicKey peerPublicKey,
Curve25519SecretKey secretKey
) internal view returns (bytes32)
```

암호화/복호화

```
// 암호화: 0x0100000000000000000000000000000000000003
// 복호화: 0x0100000000000000000000000000000000000004
// DeoxysII AEAD 알고리즘 사용
```

2.6 가스 계산 시스템

기본 가스 비용

```
gas_costs: modules::core::GasCosts {
tx_byte: 1, // 바이트당 1 가스
storage_byte: 15, // 저장 바이트당 15 가스
auth_signature: 1_000, // 서명 검증 1,000 가스
auth_multisig_signer: 1_000, // 멀티시그 서명자당 1,000 가스
callformat_x25519_deoxysii: 10_000, // 암호화 호출 10,000 가스
}
```

동적 가스 가격

```
dynamic_min_gas_price: modules::core::DynamicMinGasPrice {
enabled: true,
target_block_gas_usage_percentage: 50, // 목표 블록 가스 사용률 50%
min_price_max_change_denominator: 8, // 최대 가격 변동 1/8
}
```

2.7 상태 관리

상태 버전 관리

```
const STATE_VERSION: u32 = state_version();

const fn state_version() -> u32 {
```

```
if is_testnet() {
11 // 테스트넷 상태 버전
} else {
7 // 메인넷 상태 버전
}
}
```

마이그레이션 처리

```
fn migrate_state<C: sdk::Context>(_ctx: &C) {
let genesis = Self::genesis_state();
// 각 모듈별 파라미터 업데이트
modules::core::Module::<Config>::set_params(genesis.0.parameters);
modules::accounts::Module::set_params(genesis.1.parameters);
// ...
}
```

2.8 보안 검증

신뢰할 수 있는 서명자

```
fn trusted_signers() -> Option<TrustedSigners> {
let tps = keymanager::trusted_policy_signers();
```

```
Some(TrustedSigners {
signers: tps.signers.into_iter().map(|s| PublicKey(s.0)).collect(),
threshold: tps.threshold,
})
}
```

합의 신뢰 루트

```
fn consensus_trust_root() -> Option<TrustRoot> {
if is_testnet() {
Some(TrustRoot {
height: 24544750,
hash: "6b2b5e7990e0c8bee8035934bc52a83a16fe4bff4dbb81b1aa5a8f9409b2eafe".into(),
runtime_id: "0000000000000000000000000000000000000000000000000a6d1e3ebf60dff6c".i
chain_context: "0b91b8e4e44b2003a7c5e23ddadb5e14ef5345c0ebcb3ddcae07fa2f244cab76
})
} else {
// 메인넷 설정...
}
}
```

2.9 성능 최적화

배치 처리 최적화

- **병렬 실행**: 독립적인 트랜잭션들의 병렬 처리
- **상태 캐싱**: 자주 접근하는 상태의 메모리 캐싱
- **가스 추정**: 실행 전 가스 사용량 정확한 추정

메모리 관리

- **제로화**: 민감한 데이터 사용 후 메모리 제로화
- **스택 보호**: 스택 오버플로우 방지
- **힙 관리**: 안전한 동적 메모리 할당

결론

Sapphire의 TEE 실행 환경은 Intel SGX를 기반으로 하여 하드웨어 수준의 보안을 제공하면서도,

EVM 호환성을 유지하고 고성능 트랜잭션 처리를 가능하게 합니다.

암호화 프리컴파일과 기밀 상태 관리를 통해 기존 Ethereum 생태계와의

완벽한 호환성을 제공합니다.